

第6章 面向对象

建议学时:3-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解 Go 的"面向对象"是 C 的 struct + 函数指针的自然演化,而不是 Java/C++ 式的类体系
- 掌握 struct 定义、初始化与**工厂函数**(Go 没有构造函数)
- 把"值接收者 vs 指针接收者"彻底讲透:方法集规则、可寻址性、什么时候用哪个
- 掌握嵌入(embedding):组合优于继承,字段与方法自动提升
- 掌握接口的**隐式实现**与多态;空接口 any 的用途与代价
- 掌握类型断言与类型 switch,理解 `v, ok := x.(T)` 的安全形式
- 理解 `Stringer / error` 等内建接口的约定,掌握"组合 vs 继承"的生产判断

💡 从 C 到 Go:本章主题如何迁移

C 的"面向对象"是手工活:定义一个 struct 存数据,写一组函数 `shape_area(&s)`,再把函数指针塞进结构体模拟虚表。Linux 内核的 `file_operations`、`qsort` 的比较器,都是这套把戏。它的问题:多态表要手动维护、类型安全靠自觉、代码散布在"数据"与"操作"两个世界里。

Go 保留了 C 的素材——**struct**(没有类!)、**方法**(不过是带接收者的函数)、**接口**(不过是一组方法签名的约定)——却用三个"自然化"动作把 C 的手工活自动化:**方法语法**让"数据+操作"绑定;**嵌入**让组合表达继承关系;**隐式实现**让多态在"恰好拥有方法"时自动成立,无需任何 `implements` 声明(与 Java 的显式 `implements` 完全相反)。

对 Java/C++ 背景的读者同样要提醒:Go **没有继承、没有重载、没有构造/析构函数、没有访问修饰符 `public/private`**(用"首字母大写 = 导出"代替),"多态"也不是"继承体系里的重写",而是"接口 + 方法集"的鸭子类型。这不是残缺,是刻意:Go 团队认为继承是"耦合的捷径",组合才是"变化的边界"。

一句话主线: C 用"结构体+函数指针"手工造 OO,Go 把这三件东西语法化,并删掉了继承——你已有的 C 心智只需平移,不用重学"类"。

📦 前置知识自查

- C 的 struct、typedef、成员访问(点与箭头)
- C 的函数指针、回调、用 struct 包装函数指针表模拟多态
- 第 1 章的结构体与数组值语义、第 4 章的方法与值传递
- (对照)Java/C++ 的 class、interface、implements、继承概念——有更好,没有也能学

🚀 本章学习路线

1. **第一阶段**:§6.1-§6.3,重点攻克"值接收者 vs 指针接收者",这是 Go 新手最大的分水岭
2. **第二阶段**:§6.4-§6.6,理解嵌入与隐式接口,拿 C 的函数指针表对照着看
3. **第三阶段(实验)**:§6.7-§6.8 与章末实验:几何图形系统,把接口/嵌入/类型 switch 织进一个程序
4. **验收标准**:能不看资料说清"什么时候用值接收者、什么时候用指针接收者"

本章知识导图

- §6.1 从 C 的 struct+函数指针到 Go 的 struct+方法
- §6.2 struct 定义、初始化与工厂函数(无构造函数)
- §6.3 方法与接收者:值 vs 指针、方法集规则
- §6.4 嵌入:组合优于继承、字段与方法提升
- §6.5 接口:隐式实现、多态、空接口 any
- §6.6 类型断言与类型 switch
- §6.7 内建接口 Stringer/error 与动态分派
- §6.8 设计原则:组合 vs 继承的生产判断

§6.1 从 C 的 struct+函数指针到 Go 的 struct+方法

先回顾 C 的手工多态(你一定写过或读过):

| 示例 6-1 C 的手工多态 vs Go 的方法

```

// C 风格:结构体 + 函数指针表(示意,不能编译成 Go)
// typedef struct {
//     double (*area)(const void*);
// } ShapeOps;
//
// double circle_area(const void* self) {
//     const Circle* c = self;
//     return 3.14 * c->r * c->r;
// }
//
// ShapeOps circleOps = { circle_area };
// Shape s = { &circleOps, &circle };

// Go 风格:方法直接挂在类型上
package main

import (
    "fmt"
    "math"
)

type Circle struct {
    Radius float64
}

// 方法:func + 接收者
func (c Circle) Area() float64 {
    return math.Pi * c.Radius * c.Radius
}

func main() {
    c := Circle{Radius: 1}
    fmt.Println(c.Area()) // 3.141592653589793
}

```

差异要点:C 里"类型的方法表"要手工维护且不类型安全; `area(c)` 与 `c.area()` 只是语法糖——Go 的方法本质上仍是"带接收者的函数",但接收者让"数据与操作"天然相邻,IDE 补全、接口匹配都基于这个语法。注意:**方法必须定义在同一个包内**,不能给别人的类型(如 `int`)加方法。

§6.2 struct 定义、初始化与工厂函数

示例 6-2 struct 的多种初始化

```
package main

import "fmt"

type User struct {
    Name string
    Age  int
    City string
}

func main() {
    // 方式一:字段名初始化(推荐,顺序无关)
    u1 := User{Name: "张三", Age: 20, City: "北京"}

    // 方式二:位置初始化(不推荐,字段一变就错位)
    u2 := User{"李四", 30, "上海"}

    // 方式三:零值 + 逐个赋值
    var u3 User
    u3.Name = "王五"

    // 方式四:取指针
    u4 := &User{Name: "赵六"}

    fmt.Println(u1, u2, u3, u4.Name)
}
```

Go **没有构造函数**(没有 `User()` 这种语法)。惯用法是**工厂函数**: `func NewUser(...) *User`。它在构造时集中做校验、默认值、依赖注入——C 里你得靠"初始化函数 + 约定"。

示例 6-3 工厂函数:构造逻辑的收容所

```
package main

import (
    "errors"
    "fmt"
)

type Account struct {
    id      string
    balance float64
}

// 工厂函数:校验 + 默认值 + 返回错误
func NewAccount(id string, initial float64) (*Account, error) {
    if id == "" {
        return nil, errors.New("账号 ID 不能为空")
    }
    if initial < 0 {
        return nil, fmt.Errorf("初始余额不能为负: %v", initial)
    }
    return &Account{id: id, balance: initial}, nil
}

func main() {
    a, err := NewAccount("acc-001", 100.5)
    if err != nil {
        fmt.Println("创建失败:", err)
        return
    }
    fmt.Printf("账户 %s 余额 %.2f\n", a.id, a.balance)
}
```

§6.3 方法与接收者:值 vs 指针(必须讲透)

方法是"带接收者的函数",接收者可以是**值或指针**。选哪个,是 Go 新手最常问的问题。三条铁律:

判断问题	选择
方法会修改接收者吗?	会 → 指针接收者; 不会 → 优先值接收者
接收者很大(几百字节以上)或含可变字段(切片/map)?	指针接收者,避免每次调用整体拷贝
类型是否需要满足"指针方法集"接口?	按接口需求(见方法集规则)

示例 6-4 值接收者与指针接收者

```

package main

import "fmt"

type Counter struct {
    value int
}

// 值接收者:修改的是副本,无效!
func (c Counter) addWrong() {
    c.value++
}

// 指针接收者:修改原对象
func (c *Counter) add() {
    c.value++
}

// 值接收者:只读操作,拷贝代价小,最常用
func (c Counter) get() int {
    return c.value
}

func main() {
    c := Counter{}
    c.addWrong()
    fmt.Println("addWrong 后:", c.value) // 0:没变

    c.add()
    c.add()
    fmt.Println("add 后:", c.value) // 2

    fmt.Println(c.get()) // 2
}

```

★ 重点:方法集规则(面试与生产都常考)

值类型 `T` 的方法集包含"值接收者"方法;指针类型 `*T` 的方法集包含**全部方法**(值 + 指针接收者)。因此:

- `var c Counter`:可以调用值方法;**可寻址时**(变量、数组元素、切片元素)也能自动取址调用指针方法
- **不可寻址的值**(map 元素、函数返回值如 `NewCounter()` 返回的值、字面量)不能调用指针方法
- 接口保存的值是拷贝:若类型只有指针方法实现接口,只有 `*T` 满足接口, `T` 不满足——`var s Shape = circle{}` 编译错误, `var s Shape = &circle{}` 才合法

⚠ 易错点 6-1:map 元素"不可取址"导致无法调用指针方法

`m := map[string]Counter{}; m["a"].add()` 编译错误:map 元素不可寻址,不能取地址调用指针方法。正确做法:先取出、修改、放回: `c := m["a"]; c.add(); m["a"] = c`。同理,函数直接返回的结构体值也不能调用指针方法。

§6.4 嵌入:组合优于继承

C 用"结构体嵌套"表达 has-a,再用"函数指针表"模拟 is-a。Go 的**嵌入(embedding)**把两者合一:一个匿名字段 `type Manager struct { Employee }`,**Employee 的字段与方法自动提升**到 Manager 上,Manager 天然"继承"了 Employee 的一切——但底层仍是组合(Manager 包含一个 Employee),没有虚继承、没有菱形问题。

示例 6-5 嵌入:字段与方法提升

```
package main

import "fmt"

type Employee struct {
    Name    string
    Salary  float64
}

func (e Employee) Describe() string {
    return fmt.Sprintf("%s(月薪 %.0f)", e.Name, e.Salary)
}

// Manager 嵌入 Employee:自动获得 Name/Salary/Describe
type Manager struct {
    Employee // 嵌入:匿名字段
    TeamSize int
}

// 覆盖:Manager 自己的 Describe 优先于提升的方法
func (m Manager) Describe() string {
    return fmt.Sprintf("%s,管理 %d 人", m.Employee.Name, m.TeamSize)
}

func main() {
    m := Manager{
        Employee: Employee{Name: "张三", Salary: 20000},
        TeamSize: 8,
    }
    fmt.Println(m.Name)           // 提升字段
    fmt.Println(m.Employee.Salary) // 显式访问内层
    fmt.Println(m.Describe())     // 自己的方法胜出
}
```

⚠ 易错点 6-2:嵌入不是继承,注意字段名冲突

若 Manager 自己也定义了 `Name`, `m.Name` 解析到 Manager 自己的字段(外层遮蔽内层),访问 Employee 的要使用 `m.Employee.Name`。提升还有一层规则:**嵌入的接口**、**嵌入的指针(*Reader)**同样提升方法。方法提升与字段提升遵循同一套遮蔽规则,冲突时显式写出完整路径最安全。

§6.5 接口:隐式实现与多态

Go 的接口是一组方法签名。任何类型只要实现了全部方法,自动满足该接口——不需要、也没有 `implements` 声明。这就是与 Java 最大的对照:

Java 的写法	Go 的写法
<code>class Circle implements Shape { ... } ,必须显式声明</code>	<code>type Circle struct {...} + 恰好实现 Shape 的方法,自动成立</code>
接口 = 类层次的一部分,要提前设计	接口 = 使用方定义约定,可以事后补充,不改动实现类型
一个类只能继承一个父类(单继承)	一个类型可以实现任意多个接口(天然多接口)

示例 6-6 隐式接口:多态

```
package main

import (
    "fmt"
    "math"
)

// 接口:约定"能算面积"
type Shape interface {
    Area() float64
}

type Circle struct{ Radius float64 }

// 不需要写 implements:方法齐了就自动满足
func (c Circle) Area() float64 {
    return math.Pi * c.Radius * c.Radius
}

type Rect struct{ W, H float64 }

func (r Rect) Area() float64 { return r.W * r.H }

// 多态:一个接口,多种实现
func printArea(s Shape) {
    fmt.Printf("面积 = %.2f\n", s.Area())
}

func main() {
    shapes := []Shape{
        Circle{Radius: 1},
        Rect{W: 3, H: 4},
    }
    for _, s := range shapes {
        printArea(s) // 动态分派
    }
}
```


★ 重点:空接口 any 与"接口保存值"

`interface{}` 与它的别名 `any` 是"不承诺任何方法"的接口——任何类型都满足它,类似 C 的 `void*`。但两点不同:① `any` 保存的是**完整类型信息**(值 + 动态类型),可以安全断言回去,不是裸指针;②接口变量保存的是**拷贝**。空接口是"万能容器",代价是装箱(分配)与失去静态检查——第 10 章会看到泛型如何取代它的大部分用途。

§6.6 类型断言与类型 switch

接口里存的动态类型,可以用**类型断言**取回:

- **安全形式**: `v, ok := x.(T)`,失败时 `v` 为 T 的零值, `ok` 为 `false`,不 panic——生产必用这种
- **危险形式**: `v := x.(T)`,失败直接 panic
- **类型 switch**: `switch v := x.(type)`,按动态类型分支,还能拿到"已断言"的值

示例 6-7 类型断言与类型 switch

```
package main

import "fmt"

func describe(x any) {
    switch v := x.(type) { // 类型 switch:语法固定,注意是 type 不是类型名
    case int:
        fmt.Println("整数:", v*2)
    case string:
        fmt.Println("字符串,长度:", len(v))
    case fmt.Stringer:
        fmt.Println("有 String 方法:", v.String())
    case nil:
        fmt.Println("nil")
    default:
        fmt.Printf("未知类型 %T,值 %v\n", x, x)
    }
}

func main() {
    describe(42)
    describe("hello")
    describe(nil)
    describe(3.14)

    // 安全断言
    var a any = 100
    if n, ok := a.(int); ok {
        fmt.Println("断言成功:", n)
    } else {
        fmt.Println("断言失败")
    }
}
```

⚠ 易错点 6-3:nil 指针装入接口后,接口 != nil

`var p *MyType; var i any = p` —— `i == nil` 是 **false**!接口的非 nil 由"类型 + 值"两部分构成,一个"值为 nil 的指针"装入接口后,接口的"值部分"是 nil 但"类型部分"是 `*MyType`,整体非 nil。判断时用 `reflect.ValueOf(i).IsNil()` 或重新断言: `if p, ok := i.(*MyType); ok && p == nil`。

§6.7 内建接口:Stringer 与 error

标准库定义了几个"约定接口",实现了它们,你的类型就自动接入语言特性:

示例 6-8 Stringer:fmt 打印自动调用

```
package main

import "fmt"

type Temperature struct {
    Value float64
    Unit  string
}

// 实现 fmt.Stringer:所有 %v / %s / Println 自动调用
func (t Temperature) String() string {
    return fmt.Sprintf("%.1f%s", t.Value, t.Unit)
}

func main() {
    t := Temperature{Value: 36.5, Unit: "C"}
    fmt.Println(t)      // 36.5°C
    fmt.Printf("%v\n", t) // 36.5°C
    // 注意:%d 等非字符串动词不会调用 String()
}
```

其它经典约定接口: `error` (`Error() string`, 第 9 章)、`io.Reader` / `io.Writer` (`Read` / `Write` 各一个方法, 第 7 章)、`sort.Interface` (`Len`/`Less`/`Swap` 三个方法)。**接口越小越好**:一个方法一个接口是 Go 的审美——`io.Reader` 只有一个 `Read` 方法,却被整个生态实现。

§6.8 设计原则:组合 vs 继承的生产判断

生产判断标准(对照 C/Java 的经验):

- **is-a 是否真的需要多态替换?**是 → 接口 + 方法(Go 没有继承,用接口表达"可替换")
- **只是复用实现?**→ 嵌入(组合)。嵌入比继承更克制:提升是"委托"的语法糖,不会强加父子耦合
- **需要"多样行为"?(一个对象多种角色)**→ 实现多个接口,天然比单继承灵活
- **接口定义在哪?**→ **定义在使用方(consumer-side)**:需要什么方法就声明什么接口,实现方毫不知情——这就是"隐式实现"的最大红利,框架可以接收任何恰好满足约定的类型

🚀 工程建议:接口的粒度与命名

①接口名用"能力"命名: `Reader`、`Closer`、`Parser`,动词或"-er"结尾;②一个接口 1-3 个方法,超过就该拆;③函数参数用**最小接口**——只需要 `Read` 就别收 `*os.File`,传 `io.Reader`,测试时传 `strings.Reader` 即可;④"接受接口,返回结构体"(Accept interfaces, return structs):入参放宽到接口,返回值给具体类型,让调用方自己做决定。

本章速查表

主题	要点
没有类	Go 用 struct + 方法表达对象;没有继承/构造/析构/重载
方法语法	<code>func (r Receiver) 方法名(参数)(返回值)</code> ;同包内才能加方法
值接收者	只读、小对象; 修改无效 (副本);不可寻址值可用
指针接收者	要修改、大对象、满足指针方法集;map 元素不可用
方法集	<code>T</code> 只有值方法; <code>*T</code> 有全部;接口实现按方法集判定
工厂函数	<code>func NewX(...) (*X, error)</code> ;校验 + 默认值集中处理
嵌入	匿名字段;字段/方法自动提升;外层遮蔽内层;组合优于继承
接口	方法签名集合; 隐式实现 ,无 implements;可多接口
空接口	<code>interface{} = any</code> ;万能容器,带类型信息;有装箱代价
类型断言	<code>v, ok := x.(T)</code> 安全; <code>v := x.(T)</code> 失败 panic
类型 switch	<code>switch v := x.(type)</code> ;case 顺序敏感
nil 接口陷阱	nil 指针装入接口后接口 $\neq$ nil
内建接口	<code>Stringer(String() string)</code> 、 <code>error</code> 、 <code>io.Reader/Writer</code> 、 <code>sort.Interface</code>
生产原则	接口定义在使用方;接受接口返回结构体;小接口;组合优先

最小必会清单

- 能说出 Go 面向对象的三件套:struct、方法、接口,以及"没有"的三件套
- 能给出"值接收者 vs 指针接收者"的决策树
- 能解释方法集规则,并举例"为什么只有 `*T` 满足某接口"
- 能写出嵌入类型并解释字段/方法提升与遮蔽
- 能解释"Go 的接口为什么不需要 implements",并对照 Java
- 能写出安全类型断言与类型 switch
- 能解释 nil 接口 $\neq$ nil 指针,并给出正确判断方法
- 能说出"接口定义在使用方"的含义与好处

自测题

Q 1. 下面代码能否编译?为什么? `type T struct{}; func (t *T) M() {}; var i any = T{}; i.M()`

✅ 参考答案

不能。M 是指针接收者方法, T{} (不可寻址的临时值)不能自动取址调用它;且接口断言/调用发生在动态类型上, T 的方法集不含指针方法。应改为 `var i any = &T{}` 或把方法改为值接收者。

Q 2. 判断正误:(a) Go 支持继承与重载;(b) 值接收者方法不能修改结构体;(c) 嵌入是继承的别名;(d) 接口可以包含数据字段。

✅ 参考答案

(a) 错,无继承(嵌入是组合)无重载;(b) 对,值接收者操作副本;(c) 错,嵌入是"委托+提升"的组合语法,无 is-a 语义、无虚表;(d) 错,接口只能有方法签名。

Q 3. 为什么说"接口定义在使用方"?给出一个具体例子。

✅ 参考答案

使用方按自己的需要声明最小接口,实现方无需知道。 `func save(r io.Reader) error` 只需 Read,调用时可传文件、网络连接、strings.Reader、缓冲区——它们都没声明过"我实现了 save 的接口",只是恰好有 Read 方法。接口跟随需求演化,不约束实现。

Q 4. `var m map[string]Counter` 中执行 `m["a"].Add()` (Add 是指针方法)会怎样?正确写法是什么?

✅ 参考答案

编译错误:map 元素不可寻址。正确写法: `c := m["a"]`; `c.Add()`; `m["a"] = c`。切片元素与数组元素可寻址,map 元素不可寻址——这是 Go 少见的"同类不同权"。

Q 5. 写出一个实现 Stringer 的类型,并说明 fmt 的哪些输出形式会调用它。

✅ 参考答案

实现 `func (t T) String() string`。fmt 的 Print/Println/Printf 的 `%v`、`%s` (对 Stringer)会调用;`%d`、`%x` 等数值/其它动词不调用;`%+v` 在无 Stringer 时用字段名格式。

Q 6. 接口里保存"值为 nil 的指针"后, `i == nil` 是什么?如何正确判断?

✅ 参考答案

是 false。接口非 nil 的条件是"类型部分或值部分非零",nil 指针装入后类型部分是 `*T`,整体非 nil。判断:重新断言 `if p, ok := i.(*T); ok && p == nil` ;或 `reflect.ValueOf(i).IsNil()` (要确认非 nil 接口)。

章末实验题:几何图形系统

实验 6:几何图形系统

实验目标:实现一个图形系统:Circle 与 Rectangle 实现 Shape 接口(面积/周长),Square 通过嵌入复用 Rectangle,支持 Stringer 打印与类型 switch 分派,最后用工厂函数组装。

实验要求:

- 任务 1:定义 Shape 接口(Area/Perimeter 两个方法)(覆盖:\$6.5 接口)
- 任务 2:Circle 用**值接收者**实现两个方法(覆盖:\$6.3 接收者选择)
- 任务 3:Rectangle 用值接收者实现;Square 通过**嵌入** Rectangle 自动获得方法(覆盖:\$6.4 嵌入与提升)
- 任务 4:Square 实现 Stringer 接口,让 `fmt.Println` 输出"正方形(边长 X)"(覆盖:\$6.7 Stringer)
- 任务 5:工厂函数 `NewSquare(side)` 返回 `*Square`,含合法性校验(覆盖:\$6.2 工厂函数、\$6.3 指针接收者)
- 任务 6:主程序收集 `[]Shape`,遍历时用**类型 switch** 区分具体类型并输出(覆盖:\$6.6 类型 switch)
- 任务 7:统计总面积用接口方法(覆盖:\$6.5 多态动态分派)

实验环境:Go 1.21+, `go run main.go`, `go vet` 无警告。

第一步 定义接口与两个基础类型,跑通"多态调用面积"

第二步 加 Square:嵌入 Rectangle,验证方法提升——注意 `*Square` 是否满足 Shape(方法集规则)

第三步 实现 Stringer,观察 `fmt` 行为变化

第四步 后加工厂函数与类型 switch,完成全部要求

✔ 参考答案要点

```
// 几何图形系统:接口、嵌入、工厂函数、Stringer、类型 switch
package main

import (
    "errors"
    "fmt"
    "math"
)

// Shape 接口:图形的公共契约($6.5)
type Shape interface {
    Area() float64
    Perimeter() float64
}

// Circle:值接收者实现($6.3:纯只读方法)
type Circle struct {
    Radius float64
}

func (c Circle) Area() float64 {
    return math.Pi * c.Radius * c.Radius
}

func (c Circle) Perimeter() float64 {
    return 2 * math.Pi * c.Radius
}

// Rectangle:值接收者
type Rectangle struct {
    Width, Height float64
}

func (r Rectangle) Area() float64 {
    return r.Width * r.Height
}

func (r Rectangle) Perimeter() float64 {
    return 2 * (r.Width + r.Height)
}

// Square:嵌入 Rectangle,自动提升 Area/Perimeter($6.4 组合)
type Square struct {
    Rectangle
}

// 工厂函数:校验 + 构造($6.2)
func NewSquare(side float64) (*Square, error) {
    if side <= 0 {
        return nil, errors.New("边长必须为正")
    }
    return &Square{Rectangle{Width: side, Height: side}}, nil
}

// Stringer:fmt.Println 自动调用($6.7)
```

```

func (s *Square) String() string {
    return fmt.Sprintf("正方形(边长 %.1f)", s.Rectangle.Width)
}

// 多态:统一按接口处理($6.5 动态分派)
func totalArea(shapes []Shape) float64 {
    total := 0.0
    for _, s := range shapes {
        total += s.Area()
    }
    return total
}

func main() {
    // 多态容器:不同实现共处一列
    shapes := []Shape{
        Circle{Radius: 1.0},
        Rectangle{Width: 3, Height: 4},
    }

    // 工厂函数返回指针(指针方法集的类型)
    sq, err := NewSquare(5)
    if err != nil {
        fmt.Println("创建失败:", err)
        return
    }
    shapes = append(shapes, sq)

    // 类型 switch 分派($6.6)
    for _, s := range shapes {
        switch v := s.(type) {
        case Circle:
            fmt.Printf("圆(r=%.1f):面积 %.2f\n", v.Radius, v.Area())
        case Rectangle:
            fmt.Printf("矩形(%.1f×%.1f):面积 %.2f\n", v.Width, v.Height, v.Area())
        case *Square:
            fmt.Printf("%v:面积 %.2f\n", v, v.Area()) // %v 触发 String()
        default:
            fmt.Printf("未知图形:%T\n", s)
        }
    }

    fmt.Printf("全部图形总面积: %.2f\n", totalArea(shapes))

    // 接口断言演示:从接口里取回具体类型
    var anyShape Shape = Circle{Radius: 2}
    if c, ok := anyShape.(Circle); ok {
        fmt.Println("断言回 Circle 成功,半径", c.Radius)
    }

    // 值接收者类型满足接口,指针类型也满足:
    // Circle 满足 Shape;*Circle 同样满足(方法集规则)
    var _ Shape = Circle{}
    var _ Shape = &Circle{}

```



```
fmt.Println("方法集验证:Circle 与 *Circle 都满足 Shape")
}
```

设计说明:这个实验把本章全部知识点串成一条线:接口定义契约,值/指针接收者分别承担"只读实现"与"工厂返回",嵌入让 Square 零成本复用 Rectangle,Stringer 让类型融入 fmt,类型 switch 处理"按具体类型增强",最后的方法集验证回答"为什么 *Square 能放进 []Shape"。扩展练习:为 Circle 增加 Diameter 嵌入演示二级提升;或把 totalArea 泛型化(第 10 章),体会接口(运行时多态)与泛型(编译期多态)的分工。

下一章预告:第7章 文件 I/O。对象与接口就位,下一章走出内存:从 C 的 fopen/fread/fwrite 到 Go 的 os 包、bufio 流式读取、目录遍历、JSON 序列化——全部用标准库完成。